

I. 결정론적 1차 쿠라모토(Kuramoto) 모형의 이론 및 수치적 해석 기법

관성(질량)과 확률적 외부 잡음이 고려되지 않은 순수 결정론적 비선형 시스템인 **결정론적 1차 쿠라모토 모형(Deterministic 1st-order Kuramoto Model)**의 지배 방정식, 수학적 해석 방법 및 컴퓨터로 이산적인 해를 구하는 수치 해석 알고리즘과 참조 파이썬 코드를 정리합니다.

1. 결정론적 1차 쿠라모토 지배 방정식

결정론적 1차 쿠라모토 모형은 마찰 저항이 극도로 큰 과감쇠 한계(Overdamped limit) 상태를 기술하며, 오실레이터들의 위상 변화율(회전 속도)이 위상차에 따른 결합력에 의해 자연 시간 없이 즉각 결정된다고 가정합니다. 지배 상미분방정식(ODE)은 다음과 같이 표현됩니다.

$$\frac{d\theta_i(t)}{dt} = \omega_i + \frac{K}{N} \sum_{j=1}^N A_{ij} \sin(\theta_j(t) - \theta_i(t))$$

이 식에서 각 물리량의 정의는 다음과 같습니다:

- $\theta_i(t)$: 오실레이터 i 의 위상 (Phase)
- ω_i : 오실레이터 i 의 고유 진동수 (Intrinsic Frequency)
- K : 결합 강도 (Coupling strength)
- A_{ij} : 인접 행렬(Adjacency Matrix, 네트워크 연결 가중치 및 마스크)
- N : 전체 오실레이터의 수

2. 평균장 단순화와 자가일치 조건 (Mean-field Formulation)

결정론적 쿠라모토 모형 역시 집단적인 동기화 정도를 측정하기 위해 복소 **질서 매개변수(Order Parameter, r)**와 집단 평균 위상(ψ)을 다음과 같이 정의합니다.

$$r(t)e^{i\psi(t)} = \frac{1}{N} \sum_{j=1}^N e^{i\theta_j(t)}$$

이 관계를 복소수 평면 상에서 전개하면 $\frac{1}{N} \sum_j \sin(\theta_j - \theta_i) = r \sin(\psi - \theta_i)$ 임을 쉽게 증명할 수 있습니다. 이를 지배 방정식에 대입하면 각각의 오실레이터가 나머지 $N - 1$ 개의 진동자들과 일일이 연산하는 대신, 공통의 평균장 $r(t)$ 에만 동적으로 반응하는 평균장 방정식으로 단순화됩니다:

$$\frac{d\theta_i(t)}{dt} = \omega_i + Kr(t) \sin(\psi(t) - \theta_i(t))$$

3. 수학적 해를 구하는 방법론 (Mathematical Solution Methods)

오실레이터 수가 무한대($N \rightarrow \infty$)이고 시스템이 정상 동기화 상태(Stationary synchronized state)에 도달했다고 가정합니다. 이때 평균 위상 $\psi(t)$ 는 일정한 집단 평균 진동수 Ω 로 회전하며 ($\psi(t) = \Omega t$), 질서 매개 변수 r 은 시간에 따라 일정한 상수 값을 유지합니다.

평균 위상과 함께 회전하는 회전 좌표계를 도입하여 상대 위상차를 $\phi_i = \theta_i - \Omega t$ 로 정의하면, 방정식은 자율 상미분방정식(Autonomous ODE) 형태로 바뀝니다:

$$\frac{d\phi_i}{dt} = \omega_i - \Omega - Kr \sin \phi_i$$

이 좌표계에서 오실레이터들의 거동은 고유 진동수 크기에 따라 두 부류로 명확히 분리됩니다.

① 동기화 진동자 (Phase-locked Oscillators)

고유 진동수가 평균 진동수 근방에 위치하여 결합력 장벽보다 작거나 같은 진동자들입니다.

$$|\omega_i - \Omega| \leq Kr$$

이들은 회전 좌표계 상에서 특정 고정 위상각(Fixed point)에 수렴하여 상대 속도가 0이 됩니다 ($\frac{d\phi_i}{dt} = 0$). 위상 평형 조건은 다음과 같습니다.

$$\sin \phi_i = \frac{\omega_i - \Omega}{Kr}$$

② 비동기화 표류 진동자 (Drifting Oscillators)

고유 진동수가 결합력 한계를 초과하여 집단 속도에 결속되지 못하고 독립적으로 계속 표류하는 진동자들입니다.

$$|\omega_i - \Omega| > Kr$$

이들은 동기화 상태에 도달하지 못하고 계속 360도 회전하지만, 결합력의 영향으로 인해 특정 위상각 부근을 지날 때 회전 속도가 느려지고 반대쪽에서는 빨라집니다. 정상 상태에서 이 표류 오실레이터들이 임의의 각도 ϕ 에 존재할 정밀 확률 밀도 $n(\phi, \omega)$ 는 속도 크기에 반비례합니다:

$$n(\phi, \omega) = \frac{C}{|d\phi/dt|} = \frac{\sqrt{(\omega - \Omega)^2 - (Kr \sin \phi)^2}}{2\pi|\omega - \Omega - Kr \sin \phi|}$$

③ 쿠라모토 자가일치 관계식 및 임계값 (K_c)

동기화된 진동자들과 표류하는 진동자들의 기여도를 합산하여 복소 질서 매개변수의 자가일치 조건(Self-consistency relation) 적분식을 세우면 다음과 같습니다. 고유 진동수 분포 $g(\omega)$ 가 우함수(symmetric)이고 단봉(unimodal) 분포일 때 집단 진동수 Ω 는 분포의 중심과 일치하고, 질서 매개변수 r 은 다음 비선형 방정식을 만족합니다.

$$r = Kr \int_{-\pi/2}^{\pi/2} \cos^2 \phi g(Kr \sin \phi) d\phi$$

질서 매개변수가 비동기화 상태($r = 0$)에서 동기화 상태($r > 0$)로 상전이를 일으키는 임계 결합 강도 K_c 는 $r \rightarrow 0^+$ 극한을 취하여 적분함으로써 수학적으로 정확히 유도됩니다.

$$K_c = \frac{2}{\pi g(0)}$$

예를 들어, 고유 진동수 분포 $g(\omega)$ 가 표준 편차 γ 를 가지는 Lorentzian 분포인 경우 임계값은 $K_c = 2\gamma$ 가 되며, $K > K_c$ 일 때 생성되는 정상 상태 질서 매개변수 r 의 분기 거동은 다음과 같은 멱법칙을 따릅니다:

$$r \approx \sqrt{\frac{16}{\pi K_c^3} \frac{K - K_c}{-g''(0)}} \propto (K - K_c)^{1/2}$$

4. 이산적 수치 해석 기법 (Discrete Numerical Methods)

컴퓨터 상에서 1차 결정론적 ODE의 궤적을 전진시키는 대표적인 수치 적분 해법입니다.

① Euler 방법 (1차 전진 오일러)

가장 단순한 시간 전진 적분 기법입니다.

$$\theta_i(t + \Delta t) = \theta_i(t) + \left[\omega_i + \frac{K}{N} \sum_{j=1}^N A_{ij} \sin(\theta_j(t) - \theta_i(t)) \right] \Delta t$$

연산 속도가 극단적으로 빠르지만 수치 안정성을 담보하기 위해 보폭 Δt 를 매우 작게 설정해야 합니다.

② Runge-Kutta 4차 방법 (RK4)

상미분방정식에서 보편적으로 사용되는 높은 정확도의 이산화 기법입니다. 보폭 Δt 를 크게 잡더라도 4차 오차 정확도($\mathcal{O}(\Delta t^4)$)로 안정적으로 해 궤적을 계산합니다.

$$k_1 = f(\theta(t), t)$$

$$k_2 = f\left(\theta(t) + \frac{\Delta t}{2}k_1, t + \frac{\Delta t}{2}\right)$$

$$k_3 = f\left(\theta(t) + \frac{\Delta t}{2}k_2, t + \frac{\Delta t}{2}\right)$$

$$k_4 = f(\theta(t) + \Delta t k_3, t + \Delta t)$$

$$\theta(t + \Delta t) = \theta(t) + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

5. 이산적 수치 해석 파이썬 참조 코드

아래 코드는 4차 Runge-Kutta (RK4) 기법과 PyTorch 텐서 배치 연산을 결합하여 1차 결정론적 쿠라모토 모형의 거동을 시뮬레이션하는 파이썬 코드 예제입니다.

```

import torch
import torch.nn as nn

class DeterministicKuramoto1stSolver(nn.Module):
    def __init__(self, num_oscillators=64, dt=0.02):
        super(DeterministicKuramoto1stSolver, self).__init__()
        self.N = num_oscillators
        self.dt = dt

        # 결합 강도 매트릭스 파라미터화 (softplus 바인딩 적용)
        self.K = nn.Parameter(torch.randn(self.N, self.N) * 0.2)

    def _derivatives(self, theta, omega):
        """1계 미분계수 산출: dtheta_dt = omega + interaction"""
        K_raw = torch.nn.functional.softplus(self.K)

        # 다중 물리 블록 결합 강도 설정 (내부 인력, Inter-group 척력)
        mask_intra = torch.zeros(self.N, self.N, device=self.K.device)
        mask_intra[:self.N//2, :self.N//2] = 1.0
        mask_intra[self.N//2:, self.N//2:] = 1.0
        mask_inter = 1.0 - mask_intra
        K_active = K_raw * mask_intra * 1.2 - K_raw * mask_inter * 0.15 + (mask_intra * 0.3 - mask_inter * 0.3) * 0.1

        theta_diff = theta.unsqueeze(1) - theta.unsqueeze(2) # [B, N, N]
        interaction = torch.sum(K_active * torch.sin(theta_diff), dim=-1) / self.N # [B, N]

        dtheta_dt = omega + interaction
        return dtheta_dt

    def step_rk4(self, theta, omega):
        """
        4차 Runge-Kutta (RK4) 1회 스텝
        """
        k1 = self._derivatives(theta, omega)

        theta_2 = theta + 0.5 * self.dt * k1
        k2 = self._derivatives(theta_2, omega)

        theta_3 = theta + 0.5 * self.dt * k2
        k3 = self._derivatives(theta_3, omega)

        theta_4 = theta + self.dt * k3
        k4 = self._derivatives(theta_4, omega)

```

```
theta_next = theta + (self.dt / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
return theta_next
```

```
def simulate(self, theta_0, omega, steps=48):
    """
    결정론적 다단계 시간 적분 및 오더 파라미터 기록
    """
    batch_size = theta_0.size(0)
    theta_traj = torch.zeros(batch_size, steps, self.N, device=theta_0.device)
    r_history = torch.zeros(batch_size, steps, device=theta_0.device)

    theta = theta_0.clone()

    for step in range(steps):
        theta = self.step_rk4(theta, omega)
        theta_traj[:, step] = theta

        # 오더 파라미터 r 계산
        r = torch.abs(torch.mean(torch.exp(1j * (theta % (2 * np.pi)))), dim=-1))
        r_history[:, step] = r

    return theta_traj, r_history
```

사용 예시 데모 스크립트

```
if __name__ == "__main__":
    batch_size = 2
    solver = DeterministicKuramoto1stSolver(num_oscillators=64, dt=0.02)

    # 초기 위상 및 고유 진동수 무작위 설정
    theta_init = torch.rand(batch_size, 64) * 2 * np.pi
    omega_init = torch.randn(batch_size, 64) * 0.15

    # 48일 시뮬레이션 전개
    traj_theta, traj_r = solver.simulate(theta_init, omega_init, steps=48)

    print("결정론적 1차 오실레이터 RK4 시뮬레이션 완료:")
    print("위상 궤적 텐서 모양 (Theta Trajectory shape):", traj_theta.shape) # [2, 48, 64]
    print("최종 5일의 오더 파라미터 r 값:", traj_r[0, -5:].detach().numpy())
```

II. 확률적 1차 쿠라모토(Kuramoto) SDE 모형의 이론 및 수치적 해석 기법

관성(질량)이 없는 1차 쿠라모토 진동자계에 확률적 노이즈가 도입된 '**확률적 1차 쿠라모토 SDE 모형 (Stochastic 1st-order Kuramoto SDE Model)**'의 지배 방정식, 수학적 해석 방법 및 컴퓨터로 이산적인 해를 구하는 수치 해석 알고리즘과 참조 파이썬 코드를 정리합니다.

1. 확률적 1차 쿠라모토 SDE 지배 방정식

1차 쿠라모토 모형은 오실레이터가 상호작용할 때 지연 시간 없이 위상을 즉각적으로 조정한다고 가정합니다. 오실레이터들의 위상 상태 갱신에 외부 노이즈(시장 잡음 및 확률적 교란)를 나타내는 **표준 위너 프로세스 (Wiener Process)**를 가산 형태로 결합하면 다음과 같은 이토 확률미분방정식(Itô SDE)을 얻습니다.

$$d\theta_i(t) = \left[\omega_i + \frac{K}{N} \sum_{j=1}^N A_{ij} \sin(\theta_j(t) - \theta_i(t)) \right] dt + \sigma dW_i(t)$$

이 식에서 각 물리량의 정의는 다음과 같습니다:

- $\theta_i(t)$: 오실레이터 i 의 위상 (Phase)
- ω_i : 오실레이터 i 의 고유 진동수 (Intrinsic Frequency)
- K : 결합 강도 (Coupling strength)
- A_{ij} : 인접 행렬(Adjacency Matrix, 마스크 가중치)
- σ : 노이즈 강도 (Noise intensity)
- $W_i(t)$: 표준 위너 프로세스 (Wiener Process, Brownian Motion)로, 이산화 증분은 $dW_i(t) \approx \mathcal{N}(0, dt)$ 를 따릅니다.

상태 변수인 위상 θ_i 에 잡음이 직접 더해지는 **덧셈 노이즈(Additive Noise)** 형태이므로, 이토(Itô) 적분과 스트라토노비치(Stratonovich) 적분의 수학적 형태가 완벽히 일치하여 수식 해석이 직관적입니다.

2. 평균장 이론과 단순화 (Mean-field Approximation)

1차 쿠라모토 모형은 시스템 전체의 동기화 정도를 정량화하기 위해 ****질서 매개변수(Order Parameter, r)****와 집단의 평균 위상(ψ)을 다음과 같이 정의합니다.

$$r(t)e^{i\psi(t)} = \frac{1}{N} \sum_{j=1}^N e^{i\theta_j(t)}$$

여기서 질서 매개변수 $r(t) \in [0, 1]$ 는 0에 가까우면 모든 오실레이터가 무질서한 비동기화 상태임을 의미하고, 1에 가까우면 완벽한 동기화 상태에 도달했음을 의미합니다.

위의 질서 매개변수의 정의를 원래의 SDE 지배 방정식에 대입하여 삼각함수 공식을 정리하면, N^2 번의 상호작용 계산을 거치지 않고 각 오실레이터가 평균장에만 반응하는 형태의 **평균장 지배 방정식(Mean-field Langevin Equation)**으로 단순화할 수 있습니다:

$$d\theta_i(t) = [\omega_i + Kr(t) \sin(\psi(t) - \theta_i(t))] dt + \sigma dW_i(t)$$

이 식은 복잡한 다체 상호작용 문제를 평균장 $r(t)$ 하의 1체 운동으로 치환할 수 있어 수학적 해석 및 수치 연산 효율성을 극대화합니다.

3. 수학적 해를 구하는 방법론 (Mathematical Solution Methods)

사인 결합 항의 비선형성으로 인해 개별 궤적의 일반적인 analytical solution을 구하는 것은 불가능하지만, 통계역학적으로 다음과 같은 해석 기법을 사용합니다.

① 쿠라모토-사카구치 포커-플랑크 방정식 (Kuramoto-Sakaguchi Fokker-Planck Equation)

오실레이터의 개수가 무한대($N \rightarrow \infty$)로 수렴하는 연속체 한계(Thermodynamic limit)에서, 특정 고유진동수 ω 를 지닌 오실레이터가 시점 t 에 위상 θ 에 존재할 확률 밀도 함수 $P(\theta, \omega, t)$ 의 거동은 다음 편미분방정식을 따릅니다.

$$\frac{\partial P}{\partial t} = -\frac{\partial}{\partial \theta} [(\omega + Kr \sin(\psi - \theta)) P] + \frac{\sigma^2}{2} \frac{\partial^2 P}{\partial \theta^2}$$

이 방정식에서 정상 상태($\partial P / \partial t = 0$) 조건과 질서 매개변수 r 의 자기일치 조건(Self-consistency relation)을 유도하면 다음과 같이 동기화 상전이(Phase Transition)가 일어나는 임계 결합 강도 K_c 를 노이즈 강도 σ 의 함수로 증명할 수 있습니다. 고유 진동수 분포 $g(\omega)$ 가 로렌츠 분포(Lorentzian Distribution)이고 폭이 γ 일 때:

$$K_c = 2 \left(\gamma + \frac{\sigma^2}{2} \right)$$

즉, 시스템에 존재하는 확률적 잡음 강도 σ 가 클수록 오실레이터들을 정렬하기 위해 필요한 최소 결합 강도 K_c 의 문턱(Threshold)이 선형적으로 높아짐을 수학적으로 알 수 있습니다.

4. 이산적 수치 해석 기법 (Discrete Numerical Methods)

컴퓨터 연산을 위해 시계열 시간 축을 이산적인 시간 보폭 Δt 단위로 나누어 궤적을 전진시키는 해법들입니다.

① 오일러-마루야마 방법 (Euler-Maruyama Method)

1차 SDE 모형에 Euler-Maruyama를 적용하면 가속도가 없는 심플한 형태로 이산화됩니다.

$$\theta_i(t + \Delta t) = \theta_i(t) + \left[\omega_i + \frac{K}{N} \sum_{j=1}^N A_{ij} \sin(\theta_j(t) - \theta_i(t)) \right] \Delta t + \sigma \sqrt{\Delta t} \epsilon_i$$

여기서 $\epsilon_i \sim \mathcal{N}(0, 1)$ 은 가우시안 정규분포 난수입니다. 구현이 단순하여 연산 효율이 매우 뛰어납니다.

② 런게-쿠타-마루야마 분할 적분법 (Runge-Kutta-Maruyama Splitting Method)

결정론적 동역학 항은 고차 오차 정밀도와 어댑티브 보폭을 제공하는 **Dormand-Prince RK45** 적분기로 가상의 θ_{new} 를 유도하고, 수락된 시간 보폭 dt 에 대해 Langevin 잡음을 더해주는 분할 결합 기법입니다.

1. 결정론적 궤적 계산:

$$\theta_{new} = \text{RK45_Deterministic_Update}(\theta, \omega, dt)$$

2. Wiener Step 가산:

수락 마스크가 활성화되면 오차 요건에 맞추어 보정된 dt 크기의 난수 잡음을 갱신된 위상에 가산합니다.

$$\theta(t + dt) = \theta_{new} + \sigma \sqrt{dt} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

이 방법은 비선형 결합력의 시간 누적 오차를 고차 RK45로 모니터링하여 보폭을 유연하게 잡고, 확률적 잡음을 결합하므로 안정적이고 정확한 시뮬레이션 경로를 제공합니다.

5. 이산적 수치 해석 파이썬 참조 코드

아래 코드는 Euler-Maruyama 기법과 PyTorch 배치 텐서 연산을 이용하여 1차 확률적 쿠라모토 SDE를 해결하는 참조용 파이썬 예제 코드입니다.

```

import torch
import torch.nn as nn
import numpy as np

class StochasticKuramoto1stSolver(nn.Module):
    def __init__(self, num_oscillators=64, dt=0.01, sigma=0.05):
        super(StochasticKuramoto1stSolver, self).__init__()
        self.N = num_oscillators
        self.dt = dt

        # 결합 파라미터 및 학습 가능한 노이즈 세기 파라미터 (exp(log_sigma)로 양수 보장)
        self.K = nn.Parameter(torch.randn(self.N, self.N) * 0.2)
        self.log_sigma = nn.Parameter(torch.tensor(np.log(sigma)))

    def get_physics_parameters(self):
        sigma = torch.exp(self.log_sigma) + 0.01
        return sigma

    def _get_coupling_matrix(self):
        K_raw = torch.nn.functional.softplus(self.K)
        mask_intra = torch.zeros(self.N, self.N, device=self.K.device)
        mask_intra[:self.N//2, :self.N//2] = 1.0
        mask_intra[self.N//2:, self.N//2:] = 1.0
        mask_inter = 1.0 - mask_intra

        # 내부 결합 인력(+), 그룹 간 결합 척력(-) 적용
        K_active = K_raw * mask_intra * 1.2 - K_raw * mask_inter * 0.15 + (mask_intra * 0.3 - mask_inter * 0.3)
        return K_active

    def step_euler_maruyama(self, theta, omega):
        """
        1차 Euler-Maruyama 이산 적분 1회 수행
        """
        sigma = self.get_physics_parameters()
        K_active = self._get_coupling_matrix()

        # 상호작용 힘 계산
        theta_diff = theta.unsqueeze(1) - theta.unsqueeze(2) # [B, N, N]
        interaction = torch.sum(K_active * torch.sin(theta_diff), dim=-1) / self.N # [B, N]

        # 1차 Deterministic 위상 도함수: dtheta_dt = omega + interaction
        dtheta_dt = omega + interaction

```

```
# Langevin Stochastic Step (dt의 제곱근에 비례하는 노이즈 가산)
```

```
noise = sigma * torch.randn_like(theta) * np.sqrt(self.dt)
```

```
theta_next = theta + dtheta_dt * self.dt + noise
```

```
return theta_next
```

```
def simulate(self, theta_0, omega, steps=48):
```

```
    """
```

```
    다단계 시간 이산 시뮬레이션 수행 및 질서 매개변수 r 출력
```

```
    """
```

```
    batch_size = theta_0.size(0)
```

```
    theta_traj = torch.zeros(batch_size, steps, self.N, device=theta_0.device)
```

```
    r_history = torch.zeros(batch_size, steps, device=theta_0.device)
```

```
    theta = theta_0.clone()
```

```
    for step in range(steps):
```

```
        theta = self.step_euler_maruyama(theta, omega)
```

```
        theta_traj[:, step] = theta
```

```
        # 오더 파라미터 r 계산
```

```
        r = torch.abs(torch.mean(torch.exp(1j * (theta % (2 * np.pi)))), dim=-1))
```

```
        r_history[:, step] = r
```

```
    return theta_traj, r_history
```

```
# 사용 데모 스크립트
```

```
if __name__ == "__main__":
```

```
    batch_size = 2
```

```
    solver = StochasticKuramoto1stSolver(num_oscillators=64, dt=0.02, sigma=0.06)
```

```
# 초기 상태 생성
```

```
theta_init = torch.rand(batch_size, 64) * 2 * np.pi
```

```
omega_init = torch.rand(batch_size, 64) * 0.15
```

```
# 48일(스텝) 시뮬레이션
```

```
traj_theta, traj_r = solver.simulate(theta_init, omega_init, steps=48)
```

```
print("1차 SDE 오실레이터 시뮬레이션 완료:")
```

```
print("위상 궤적 텐서 모양 (Theta Trajectory shape):", traj_theta.shape) # [2, 48, 64]
```

```
print("오더 파라미터 변동 추이 (최종 5 스텝):", traj_r[0, -5:].detach().numpy())
```

III. 결정론적 2차 쿠라모토(Kuramoto) 모형의 이론 및 수치적 해석 기법

질량(관성)과 감쇠(저항)가 존재하며 확률적 노이즈가 배제된 결정론적 상미분 동역학계 모델인 '결정론적 2차 쿠라모토 모형(Deterministic 2nd-order Kuramoto Model)'의 지배 방정식, 수학적 해석 방법 및 컴퓨터로 이산적인 해를 구하는 수치 해석 알고리즘과 참조 파이썬 코드를 상세히 정리합니다.

1. 결정론적 2차 쿠라모토 지배 방정식

2차 쿠라모토 모형은 개별 오실레이터에 물리적인 회전 질량(관성, m)과 저항 역할을 하는 감쇠 계수(마찰, α)를 추가하여 뉴턴 역학적인 가속도 운동을 기술합니다. 지배 방정식은 다음과 같은 2계 상미분방정식(2nd-order ODE)으로 정의됩니다.

$$m\ddot{\theta}_i(t) + \alpha\dot{\theta}_i(t) = \omega_i + \frac{K}{N} \sum_{j=1}^N A_{ij} \sin(\theta_j(t) - \theta_i(t))$$

이 식에서 각 물리량의 정의는 다음과 같습니다:

- $\theta_i(t)$: 오실레이터 i 의 위상 (Phase)
- $v_i(t) = \dot{\theta}_i(t)$: 오실레이터 i 의 각속도 (Angular Velocity)
- m : 오실레이터의 질량 (관성 크기, Inertia)
- α : 감쇠 계수 (마찰/저항 계수, Damping coefficient)
- ω_i : 오실레이터 i 의 고유 진동수 (Intrinsic Frequency)
- K : 결합 강도 (Coupling strength)
- A_{ij} : 인접 행렬(Adjacency Matrix, 마스크 가중치)
- N : 전체 오실레이터 수

2. 상태 공간 표현식 및 평균장 단순화 (State-Space & Mean-Field)

컴퓨터 상에서 수치 연산을 효과적으로 수행하기 위해 속도 상태 변수 $v_i(t) = \dot{\theta}_i(t)$ 를 도입하여 2계 ODE를 연립 1계 ODE 시스템으로 분리합니다.

$$\frac{d\theta_i(t)}{dt} = v_i(t)$$

$$\frac{dv_i(t)}{dt} = \frac{1}{m} \left[\omega_i - \alpha v_i(t) + \frac{K}{N} \sum_{j=1}^N A_{ij} \sin(\theta_j(t) - \theta_i(t)) \right]$$

또한, 집단의 평균 흐름을 대변하는 복소 **질서 매개변수(Order Parameter, r)**와 집단 평균 위상(ψ)을 다음과 같이 정의합니다:

$$r(t)e^{i\psi(t)} = \frac{1}{N} \sum_{j=1}^N e^{i\theta_j(t)}$$

이를 결합하면 각 오실레이터의 질량 가속 운동은 공유하는 평균장 $r(t)$ 에만 동기화 힘을 받는 형태로 단순화 됩니다:

$$m\ddot{\theta}_i(t) + \alpha\dot{\theta}_i(t) = \omega_i + Kr(t) \sin(\psi(t) - \theta_i(t))$$

3. 수학적 해를 구하는 방법론 (Mathematical Solution Methods)

오실레이터의 개수가 충분히 많고 시스템이 정상 상태에 도달해 평균 위상 $\psi(t)$ 가 일정한 평균 진동수 Ω 로 회전하며 질서 매개변수 r 이 일정한 상수로 안착했다고 가정합니다.

평균 위상의 속도로 회전하는 동기식 회전 좌표계를 적용하여 상대 위상차를 $\phi_i = \theta_i - \Omega t$ 로 두면 다음과 같은 지배 방정식이 유도됩니다:

$$m\ddot{\phi}_i + \alpha\dot{\phi}_i = \omega_i - \alpha\Omega - Kr \sin \phi_i$$

① 동기화 진동자 (Phase-locked Oscillators)

상대 속도와 상대 가속도가 완전히 0이 되어 특정 평형 위상각에 고정되는 오실레이터들입니다 ($\dot{\phi}_i = \ddot{\phi}_i = 0$). 위상 고정 평형 조건식은 다음과 같습니다.

$$\sin \phi_i = \frac{\omega_i - \alpha\Omega}{Kr}$$

이 평형 실근이 존재하기 위한 필요충분조건은 고유진동수의 편차가 결합력보다 작아야 함을 의미합니다:

$$|\omega_i - \alpha\Omega| \leq Kr$$

이때 1차 모델과 달리 2차 모델은 동일 범위 내에 두 개의 평형 상태(안정 노드와 안장점)가 공존하며, $\cos \phi_i > 0$ 을 만족하는 위상 각도만 국소적 안정성(Local stability)을 확보합니다.

② 비동기화 진동자 및 표류 (Drifting Oscillators)

고유진동수 편차가 결합 한계를 벗어나 동기화에 잡히지 않고 계속 회전하는 진동자들입니다 ($|\omega_i - \alpha\Omega| > Kr$). 이들은 마찰력과 질량 관성의 이중 제어를 받으며 끊임없이 주파수가 요동치는 표류 운동을 전개합니다.

③ 1차 불연속 상전이 및 히스테리시스 이력 현상 (Hysteresis)

2차 쿠라모토 모델의 가장 핵심적인 수학적·물리적 특성은 1차 모델의 연속 상전이와 달리 **1차 불연속 상전이 (1st-order Discontinuous Phase Transition)** 와 **히스테리시스(이력 현상)** 를 유발한다는 점입니다.

- **불연속 상전이:** 결합 강도 K 를 점진적으로 늘릴 때, 동기화 지표 r 이 0에서 완만하게 상승하지 않고 임계점 K_c 에 도달하는 순간 갑자기 유한한 동기화 값으로 **불연속적인 점프(Discontinuous Jump)** 를 일으킵니다.
- **히스테리시스 루프:** 이미 동기화된 시스템($r > 0$)의 결합도를 다시 약화시킬 때, 결합도가 K_c 보다 작아 지더라도 동기화가 즉시 파괴되지 않고 더 낮은 하한 임계점 K_c^- 까지 동기화 상태가 잔존합니다. 즉, $K \in [K_c^-, K_c]$ 구간에서 시스템은 과거의 경로에 따라 동기화 상태와 비동기화 상태 중 하나로 결정되는 다중 안정성(Multistability) 및 시간적 이력을 가집니다. 이 관성(질량)에 의한 히스테리시스 특성은 주식 시장에서 정보 반영이 지연되며 급등락 추세가 지속되는 오버슈팅 현상을 설명하는 물리적 근거가 됩니다.

4. 이산적 수치 해석 기법 (Discrete Numerical Methods)

2차 상미분방정식을 이산적으로 해결하기 위해 상태 공간 텐서 $\vec{x}(t) = [\vec{\theta}(t), \vec{v}(t)]^T$ 를 구성하고 시간 스텝 Δt 단위로 전진시킵니다.

① Euler 방법 (1차 연립 적분)

가장 원초적인 이산화 스텝 기법입니다.

$$\theta_i(t + \Delta t) = \theta_i(t) + v_i(t)\Delta t$$

$$v_i(t + \Delta t) = v_i(t) + \frac{1}{m} [\omega_i - \alpha v_i(t) + \text{coupling}_i(t)] \Delta t$$

② Runge-Kutta 4차 방법 (RK4)

상태 공간 연립 미분 벡터 방정식 $\vec{x}' = \vec{f}(\vec{x}, t)$ 에 대해 오차 차수 $\mathcal{O}(\Delta t^4)$ 를 제공하는 표준 수치해석 기법입니다.

$$\vec{k}_1 = \vec{f}(\vec{x}(t), t)$$

$$\vec{k}_2 = \vec{f}\left(\vec{x}(t) + \frac{\Delta t}{2}\vec{k}_1, t + \frac{\Delta t}{2}\right)$$

$$\vec{k}_3 = \vec{f}\left(\vec{x}(t) + \frac{\Delta t}{2}\vec{k}_2, t + \frac{\Delta t}{2}\right)$$

$$\vec{k}_4 = \vec{f}(\vec{x}(t) + \Delta t\vec{k}_3, t + \Delta t)$$

$$\vec{x}(t + \Delta t) = \vec{x}(t) + \frac{\Delta t}{6}(\vec{k}_1 + 2\vec{k}_2 + 2\vec{k}_3 + \vec{k}_4)$$

5. 이산적 수치 해석 파이썬 참조 코드

아래 코드는 PyTorch 배치 연산과 4차 Runge-Kutta (RK4) 수치 적분을 이용하여 결정론적 2차 쿠라모토 모형의 궤적을 산출하는 단독 구동용 파이썬 클래스 예제입니다.


```

import torch
import torch.nn as nn

class DeterministicKuramoto2ndSolver(nn.Module):
    def __init__(self, num_oscillators=64, dt=0.02, mass=1.5, damping=0.3):
        super(DeterministicKuramoto2ndSolver, self).__init__()
        self.N = num_oscillators
        self.dt = dt

        # 학습 및 최적화 가능한 물리 파라미터 제약화 설정
        self.raw_mass = nn.Parameter(torch.tensor(mass))
        self.raw_damping = nn.Parameter(torch.tensor(damping))
        self.K = nn.Parameter(torch.randn(self.N, self.N) * 0.2)

    def get_physics_parameters(self):
        m = torch.nn.functional.softplus(self.raw_mass) + 0.5
        alpha = torch.sigmoid(self.raw_damping) * 0.45 + 0.05
        return m, alpha

    def _derivatives(self, theta, v, omega):
        """2차 연립 미분계수 산출: dtheta_dt = v, dv_dt = (omega - alpha*v + interaction)/m"""
        m, alpha = self.get_physics_parameters()
        K_raw = torch.nn.functional.softplus(self.K)

        # 다중 물리 내부/그룹간 결합 설정
        mask_intra = torch.zeros(self.N, self.N, device=self.K.device)
        mask_intra[:self.N//2, :self.N//2] = 1.0
        mask_intra[self.N//2:, self.N//2:] = 1.0
        mask_inter = 1.0 - mask_intra
        K_active = K_raw * mask_intra * 1.2 - K_raw * mask_inter * 0.15 + (mask_intra * 0.3 - mask_inter * 0.3)

        theta_diff = theta.unsqueeze(1) - theta.unsqueeze(2) # [B, N, N]
        interaction = torch.sum(K_active * torch.sin(theta_diff), dim=-1) / self.N # [B, N]

        # dtheta_dt 및 dv_dt 반환
        dtheta_dt = v
        dv_dt = (omega - alpha * v + interaction) / m
        return dtheta_dt, dv_dt

    def step_rk4(self, theta, v, omega):
        """
        위상과 속도의 연립 4차 Runge-Kutta (RK4) 이산화 스텝
        """

```

```

k1_t, k1_v = self._derivatives(theta, v, omega)

t2 = theta + 0.5 * self.dt * k1_t
v2 = v + 0.5 * self.dt * k1_v
k2_t, k2_v = self._derivatives(t2, v2, omega)

t3 = theta + 0.5 * self.dt * k2_t
v3 = v + 0.5 * self.dt * k2_v
k3_t, k3_v = self._derivatives(t3, v3, omega)

t4 = theta + self.dt * k3_t
v4 = v + self.dt * k3_v
k4_t, k4_v = self._derivatives(t4, v4, omega)

theta_next = theta + (self.dt / 6.0) * (k1_t + 2*k2_t + 2*k3_t + k4_t)
v_next      = v + (self.dt / 6.0) * (k1_v + 2*k2_v + 2*k3_v + k4_v)

return theta_next, v_next

def simulate(self, theta_0, v_0, omega, steps=48):
    """
    결정론적 다단계 시간 연립 시뮬레이션 수행 및 질서 매개변수 r 출력
    """
    batch_size = theta_0.size(0)
    theta_traj = torch.zeros(batch_size, steps, self.N, device=theta_0.device)
    v_traj = torch.zeros(batch_size, steps, self.N, device=theta_0.device)
    r_history = torch.zeros(batch_size, steps, device=theta_0.device)

    theta = theta_0.clone()
    v = v_0.clone()

    for step in range(steps):
        theta, v = self.step_rk4(theta, v, omega)
        theta_traj[:, step] = theta
        v_traj[:, step] = v

        # 오더 파라미터 r 계산
        r = torch.abs(torch.mean(torch.exp(1j * (theta % (2 * np.pi)))), dim=-1))
        r_history[:, step] = r

    return theta_traj, v_traj, r_history

```

데모 실행부

```

if __name__ == "__main__":
    batch_size = 2
    solver = DeterministicKuramoto2ndSolver(num_oscillators=64, dt=0.02, mass=1.8, damping=0.25)

    # 초기 상태 및 속도 설정
    theta_init = torch.rand(batch_size, 64) * 2 * np.pi
    v_init = torch.randn(batch_size, 64) * 0.05
    omega_init = torch.randn(batch_size, 64) * 0.1

    # 48 스텝 시뮬레이션 전개
    traj_theta, traj_v, traj_r = solver.simulate(theta_init, v_init, omega_init, steps=48)

    m_val, alpha_val = solver.get_physics_parameters()
    print(f"물리 파라미터 확인 -> 질량: {m_val.item():.4f}, 감쇠비: {alpha_val.item():.4f}")
    print("결정론적 2차 오실레이터 연립 RK4 시뮬레이션 완료:")
    print("위상 궤적 텐서 모양 (Theta Trajectory shape):", traj_theta.shape) # [2, 48, 64]
    print("속도 궤적 텐서 모양 (Velocity Trajectory shape):", traj_v.shape) # [2, 48, 64]
    print("최종 5일의 오더 파라미터 r 값:", traj_r[0, -5:].detach().numpy())

```

IV. 확률적 2차 쿠라모토(Kuramoto) SDE 모형의 이론 및 수치적 해석 기법

질량(관성)과 감쇠(저항)가 있는 2차 쿠라모토 진동자계에 확률적 노이즈가 도입된 '**확률적 2차 쿠라모토 SDE 모형(Stochastic 2nd-order Kuramoto SDE Model)**'의 지배 방정식, 수학적 해석 방법 및 컴퓨터로 이산적인 해를 구하는 수치 해석 알고리즘과 참조 파이썬 코드를 상세히 정리합니다.

1. 확률적 2차 쿠라모토 SDE 지배 방정식

2차 쿠라모토 모형은 개별 오실레이터의 회전 운동에 관성(Inertia)을 나타내는 **질량(m)**과 저항을 나타내는 **감쇠 계수(α)**를 고려합니다. 여기에 외부 교란 및 시장의 무작위 변동성을 묘사하는 **가우시안 백색 잡음(Gaussian White Noise)**을 가속력(Force)단에 결합하면 아래와 같은 2계 확률미분방정식(Langevin SDE)을 얻습니다.

$$m\ddot{\theta}_i(t) + \alpha\dot{\theta}_i(t) = \omega_i + \frac{K}{N} \sum_{j=1}^N A_{ij} \sin(\theta_j(t) - \theta_i(t)) + \sigma\xi_i(t)$$

이 식에서 각 물리량의 정의는 다음과 같습니다:

- $\theta_i(t)$: 오실레이터 i 의 위상 (Phase)
- $v_i(t) = \dot{\theta}_i(t)$: 오실레이터 i 의 각속도 (Angular Velocity)
- m : 오실레이터의 질량 (관성량, Inertia)
- α : 감쇠 계수 (마찰/저항력, Damping coefficient)
- ω_i : 오실레이터 i 의 고유 진동수 (Intrinsic Frequency)
- K : 결합 강도 (Coupling strength)
- A_{ij} : 인접 행렬(Adjacency Matrix, 마스크 가중치)
- σ : 노이즈 강도 (Noise intensity)
- $\xi_i(t)$: 평균이 0이고 상관관계가 없는 독립 가우시안 백색 잡음 (Gaussian White Noise)

$$\langle \xi_i(t) \rangle = 0, \quad \langle \xi_i(t) \xi_j(t') \rangle = \delta_{ij} \delta(t - t')$$

2. 상태 공간 방정식 (State-Space Representation)

컴퓨터 상에서 수치 적분을 수월하게 수행하기 위해, 위의 2계 SDE를 2개의 연립 1계 이토 확률미분방정식(Itô SDE) 시스템으로 변환합니다. 속도 변수 $v_i(t) = \dot{\theta}_i(t)$ 를 도입하면 다음과 같이 상태 공간 방정식으로 기술됩니다.

$$\begin{aligned} d\theta_i(t) &= v_i(t)dt \\ dv_i(t) &= \frac{1}{m} \left[\omega_i - \alpha v_i(t) + \frac{K}{N} \sum_{j=1}^N A_{ij} \sin(\theta_j(t) - \theta_i(t)) \right] dt + \frac{\sigma}{m} dW_i(t) \end{aligned}$$

여기서 $W_i(t)$ 는 표준 위너 프로세스(Wiener Process, Brownian Motion)이며, $dW_i(t) = W_i(t + dt) - W_i(t) \approx \mathcal{N}(0, dt)$ 의 확률적 증분을 가집니다.

노이즈가 상태 변수(θ, v)에 곱해지는 형태가 아닌 가산 형태로 더해지는 **덧셈 노이즈(Additive Noise)** 형식이므로, Itô 적분과 Stratonovich 적분의 확률 기하학적 정의 차이가 발생하지 않고 수식이 일치하는 편리함을 제공합니다.

3. 수학적 해를 구하는 방법론 (Mathematical Solution Methods)

비선형 사인 함수 커플링 항 $\sin(\theta_j - \theta_i)$ 과 다체 상호작용의 비선형성으로 인해 이 방정식의 일반적인 대수적 분석해(Closed-form analytical solution)를 구하는 것은 불가능합니다. 따라서 다음과 같은 거시적 통계학적 접근과 컴퓨터 수치해석적 접근을 활용하여 해를 도출합니다.

① 크레이머스-포커-플랑크 방정식 (Kramers-Fokker-Planck Equation)

개별 진동자의 미시적 경로를 구하는 대신, 진동자계 전체의 상태 확률 분포 $P(\vec{\theta}, \vec{v}, t)$ 의 거동을 분석합니다. 상태 공간에서의 확률 흐름의 보존 법칙(Continuity Equation)을 유도하면 다음과 같은 편미분방정식을 얻습니다.

$$\frac{\partial P}{\partial t} = - \sum_{i=1}^N v_i \frac{\partial P}{\partial \theta_i} - \sum_{i=1}^N \frac{\partial}{\partial v_i} \left[F_i(\vec{\theta}, \vec{v}) P \right] + \frac{\sigma^2}{2m^2} \sum_{i=1}^N \frac{\partial^2 P}{\partial v_i^2}$$

여기서 $F_i(\vec{\theta}, \dots) = \frac{1}{m} (\omega_i - \alpha v_i + \text{coupling}_i)$ 입니다. 통계 열역학 한계($N \rightarrow \infty$)에서 위 편미분방정식을 수치적으로 풀거나 평균장 이론(Mean-field Theory)을 적용해 동기화가 깨지는 임계 온도(임계 노이즈 세기)와 히스테리시스 이력 거동을 정량적으로 증명합니다.

② 이산적 수치 시뮬레이션 (Discrete Numerical Integration)

현실적인 금융 시계열 예측과 같은 신경망 파이프라인 내부에서는 유한한 N 개의 오실레이터 궤적을 컴퓨터 상에서 한 단계씩 전진시키는 이산적 시간 전진(Discrete Time Stepping) 기법을 사용하여 궤적 해를 구합니다.

4. 이산적 수치 해석 기법 (Discrete Numerical Methods)

SDE의 해 경로를 근사하기 위한 대표적인 이산화 알고리즘입니다.

① 오일러-마루야마 방법 (Euler-Maruyama Method)

확률미분방정식에서 가장 기본적으로 활용되는 1차 적분법입니다. Deterministic 항에는 일반 Euler 스텝을, Stochastic 항에는 위너 프로세스의 독립 이산화 증분 $\Delta W_i = \sqrt{\Delta t} \epsilon_i$ (단, $\epsilon_i \sim \mathcal{N}(0, 1)$)을 적용합니다.

$$\theta_i(t + \Delta t) = \theta_i(t) + v_i(t) \Delta t$$

$$v_i(t + \Delta t) = v_i(t) + \frac{1}{m} \left[\omega_i - \alpha v_i(t) + \frac{K}{N} \sum_{j=1}^N A_{ij} \sin(\theta_j(t) - \theta_i(t)) \right] \Delta t + \frac{\sigma}{m} \sqrt{\Delta t} \epsilon_i$$

- **수렴도:** 강한 수렴 차수(Strong convergence order)는 0.5, 약한 수렴 차수(Weak convergence order)는 1.0을 보입니다. 구조가 간단하여 빠른 시뮬레이션에 적합하지만, Δt 가 크거나 비선형성이 결합될 때 수치 폭주(Explosion)가 일어날 위험이 높습니다.

② 런게-쿠타-마루야마 분할 적분법 (Runge-Kutta-Maruyama Splitting Method)

결결론적(Deterministic) 상미분 파트에는 고차 및 어댑티브 제어가 가능한 **Dormand-Prince RK45** 공식을 적용하고, 승인된 시간 보폭 dt 만큼 Langevin 확률 파트를 기하학적으로 합성하는 하이브리드 연립 분할 기법입니다.

1. 상태 미분계수 산출:

$$\vec{k}_t, \vec{k}_v = \text{Derivatives}(\vec{\theta}, \vec{v}, \vec{\omega})$$

2. RK45 적분 (가상 결정론적 업데이트):

표준 Dormand-Prince 계수 행렬(a_{ij}, b_i)을 적용하여 한 단계 진행된 가상의 결정론적 최종 상태인 $\vec{\theta}_{new}, \vec{v}_{new}$ 를 유도합니다.

3. 오차 노름 검증 및 보폭 dt 가감:

위상 오차와 속도 오차의 가중 평균 노름이 오차 한계($rtol, atol$) 이하인지 확인하여 보폭 dt 를 갱신합니다.

4. Langevin 위너 프로세스 합성 (stochastic update):

적분이 수락되면 가속 노이즈 변동량을 속도에 누적 반영합니다.

$$\vec{\theta}(t + dt) = \vec{\theta}_{new}$$

$$\vec{v}(t + dt) = \vec{v}_{new} + \frac{\sigma}{m} \sqrt{dt} \vec{\epsilon}, \quad \vec{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$$

이 기법은 비선형 커플링으로 인한 결정론적 수치 에러를 고차 런게쿠타로 억제하면서 확률적 Langevin 운동을 강건하게 주입할 수 있어 SDE 수치 안정성을 획기적으로 개선합니다.

5. 이산적 수치 해석 파이썬 참조 코드

아래 코드는 이산적 Euler-Maruyama 기법과 PyTorch 텐서 배치를 활용한 하이브리드 RK45 SDE 시뮬레이션 방식을 단독 스크립트로 동작할 수 있게 구현한 참조용 파이썬 예제 코드입니다.

```

import torch
import torch.nn as nn
import numpy as np

class StochasticKuramoto2ndSolver(nn.Module):
    def __init__(self, num_oscillators=64, dt=0.01, sigma=0.05, mass=1.5, damping=0.3):
        super(StochasticKuramoto2ndSolver, self).__init__()
        self.N = num_oscillators
        self.dt = dt

        # 물리 파라미터 (미분 연산을 위해 소프트플러스와 시그모이드 바인딩 제약 적용)
        self.raw_mass = nn.Parameter(torch.tensor(mass))
        self.raw_damping = nn.Parameter(torch.tensor(damping))
        self.K = nn.Parameter(torch.randn(self.N, self.N) * 0.2)
        self.log_sigma = nn.Parameter(torch.tensor(np.log(sigma)))

    def get_physics_parameters(self):
        m = torch.nn.functional.softplus(self.raw_mass) + 0.5
        alpha = torch.sigmoid(self.raw_damping) * 0.45 + 0.05
        sigma = torch.exp(self.log_sigma) + 0.01
        return m, alpha, sigma

    def _get_coupling_matrix(self):
        # 내부 인력 결합 마스크 설정
        K_raw = torch.nn.functional.softplus(self.K)
        mask_intra = torch.zeros(self.N, self.N, device=self.K.device)
        mask_intra[:self.N//2, :self.N//2] = 1.0
        mask_intra[self.N//2:, self.N//2:] = 1.0
        mask_inter = 1.0 - mask_intra

        # 결합 강도 행렬 구축 (내부 결합 (+), 교차 결합 (-))
        K_active = K_raw * mask_intra * 1.2 - K_raw * mask_inter * 0.15 + (mask_intra * 0.3 - m
        return K_active

    def step_euler_maruyama(self, theta, v, omega):
        """
        Euler-Maruyama 이산 적분 방식 (1회 스텝)
        """
        m, alpha, sigma = self.get_physics_parameters()
        K_active = self._get_coupling_matrix()

        # 커플링 힘 계산
        theta_diff = theta.unsqueeze(1) - theta.unsqueeze(2) # [B, N, N]

```

```
interaction = torch.sum(K_active * torch.sin(theta_diff), dim=-1) / self.N # [B, N]
```

```
# 미분식
```

```
dtheta_dt = v
```

```
dv_dt = (omega - alpha * v + interaction) / m
```

```
# Euler Step
```

```
theta_next = theta + dtheta_dt * self.dt
```

```
# Langevin Stochastic Step
```

```
noise_v = (sigma * torch.randn_like(v) * np.sqrt(self.dt)) / m
```

```
v_next = v + dv_dt * self.dt + noise_v
```

```
return theta_next, v_next
```

```
def simulate(self, theta_0, v_0, omega, steps=48):
```

```
    """
```

```
    전체 시간 단계 시뮬레이션 및 오더 파라미터 반환
```

```
    """
```

```
    batch_size = theta_0.size(0)
```

```
    theta_traj = torch.zeros(batch_size, steps, self.N, device=theta_0.device)
```

```
    v_traj = torch.zeros(batch_size, steps, self.N, device=theta_0.device)
```

```
    r_history = torch.zeros(batch_size, steps, device=theta_0.device)
```

```
    theta = theta_0.clone()
```

```
    v = v_0.clone()
```

```
    for step in range(steps):
```

```
        theta, v = self.step_euler_maruyama(theta, v, omega)
```

```
        theta_traj[:, step] = theta
```

```
        v_traj[:, step] = v
```

```
        # 질서 매개변수 r 계산 (복소 공간 상의 지수 위상값들의 평균 노름)
```

```
        r = torch.abs(torch.mean(torch.exp(1j * (theta % (2 * np.pi)))), dim=-1))
```

```
        r_history[:, step] = r
```

```
    return theta_traj, v_traj, r_history
```

```
# 사용 예시 데모 코드
```

```
if __name__ == "__main__":
```

```
    batch_size = 2
```

```
    solver = StochasticKuramoto2ndSolver(num_oscillators=64, dt=0.02, sigma=0.08)
```

```
# 초기 임의 상태 생성
```



```
theta_init = torch.rand(batch_size, 64) * 2 * np.pi
v_init = torch.randn(batch_size, 64) * 0.1
omega_init = torch.randn(batch_size, 64) * 0.2

# 48일의 궤적 계산
traj_theta, traj_v, traj_r = solver.simulate(theta_init, v_init, omega_init, steps=48)

print("오실레이터 시뮬레이션 완료:")
print("위상 궤적 텐서 모양 (Theta Trajectory shape):", traj_theta.shape) # [2, 48, 64]
print("속도 궤적 텐서 모양 (Velocity Trajectory shape):", traj_v.shape) # [2, 48, 64]
print("오더 파라미터 변동 추이 (최종 5 스텝):", traj_r[0, -5:].detach().numpy())
```